

Optimal Polynomial-Time Compression for Boolean Max CSP

Citation for published version (APA):

Jansen, B. M. P., & Włodarczyk, M. (2024). Optimal Polynomial-Time Compression for Boolean Max CSP. *ACM Transactions on Computation Theory*, 16(1), Article 4. <https://doi.org/10.1145/3624704>

Document license:

CC BY

DOI:

[10.1145/3624704](https://doi.org/10.1145/3624704)

Document status and date:

Published: 01/03/2024

Document Version:

Publisher's PDF, also known as Version of Record (includes final page, issue and volume numbers)

Please check the document version of this publication:

- A submitted manuscript is the version of the article upon submission and before peer-review. There can be important differences between the submitted version and the official published version of record. People interested in the research are advised to contact the author for the final version of the publication, or visit the DOI to the publisher's website.
- The final author version and the galley proof are versions of the publication after peer review.
- The final published version features the final layout of the paper including the volume, issue and page numbers.

[Link to publication](#)

General rights

Copyright and moral rights for the publications made accessible in the public portal are retained by the authors and/or other copyright owners and it is a condition of accessing publications that users recognise and abide by the legal requirements associated with these rights.

- Users may download and print one copy of any publication from the public portal for the purpose of private study or research.
- You may not further distribute the material or use it for any profit-making activity or commercial gain
- You may freely distribute the URL identifying the publication in the public portal.

If the publication is distributed under the terms of Article 25fa of the Dutch Copyright Act, indicated by the "Taverne" license above, please follow below link for the End User Agreement:

www.tue.nl/taverne

Take down policy

If you believe that this document breaches copyright please contact us at:

openaccess@tue.nl

providing details and we will investigate your claim.



Optimal Polynomial-Time Compression for Boolean Max CSP

BART M. P. JANSEN and MICHAŁ WŁODARCZYK, Eindhoven University of Technology, The Netherlands

In the Boolean maximum constraint satisfaction problem—MAX CSP(Γ)—one is given a collection of weighted applications of constraints from a finite constraint language Γ , over a common set of variables, and the goal is to assign Boolean values to the variables so that the total weight of satisfied constraints is maximized. There exists a concise dichotomy theorem providing a criterion on Γ for the problem to be polynomial-time solvable and stating that otherwise, it becomes NP-hard. We study the NP-hard cases through the lens of kernelization and provide a complete characterization of MAX CSP(Γ) with respect to the optimal compression size. Namely, we prove that MAX CSP(Γ) parameterized by the number of variables n is either polynomial-time solvable, or there exists an integer $d \geq 2$ depending on Γ , such that:

- (1) An instance of MAX CSP(Γ) can be compressed into an equivalent instance with $O(n^d \log n)$ bits in polynomial time,
- (2) MAX CSP(Γ) does not admit such a compression to $O(n^{d-\varepsilon})$ bits unless $\text{NP} \subseteq \text{co-NP/poly}$.

Our reductions are based on interpreting constraints as multilinear polynomials combined with the framework of “constraint implementations”, formerly used in the context of APX-hardness. As another application of our reductions, we reveal tight connections between optimal running times for solving MAX CSP(Γ). More precisely, we show that obtaining a running time of the form $O(2^{(1-\varepsilon)n})$ for particular classes of MAX CSPs is as hard as breaching this barrier for MAX d -SAT for some d .

CCS Concepts: • **Theory of computation** → **Parameterized complexity and exact algorithms**; • **Mathematics of computing** → *Combinatorial optimization*;

Additional Key Words and Phrases: Constraint satisfaction problem, kernelization, exponential-time algorithms

An extended abstract of this work appeared in the Proceedings of the 28th Annual European Symposium on Algorithms, ESA 2020.

This project has received funding from the European Research Council (ERC) under the European Union’s Horizon 2020 research and innovation programme (grant agreement No. 803421, ReduceSearch). The second author was supported by the Foundation for Polish Science (FNP).



Authors’ address: B. M. P. Jansen and M. Włodarczyk, Eindhoven University of Technology, P.O. Box 513, Eindhoven 5600 MB, The Netherlands; e-mails: {b.m.p.jansen, m.wlodarczyk}@tue.nl.



This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2024 Copyright held by the owner/author(s).

1942-3454/2024/03-ART4

<https://doi.org/10.1145/3624704>

ACM Reference format:

Bart M. P. Jansen and Michał Włodarczyk. 2024. Optimal Polynomial-Time Compression for Boolean Max CSP. *ACM Trans. Comput. Theory* 16, 1, Article 4 (March 2024), 20 pages.
<https://doi.org/10.1145/3624704>

1 INTRODUCTION

Background and motivation. The framework of **constraint satisfaction problems (CSPs)** allows the computational complexity of a large class of problems to be studied through a common lens [11]. A typical instance of such a problem asks whether it is possible to assign each of the variables $x_1, \dots, x_n$ a value from a finite domain D , such that a given list of constraint applications is satisfied. A constraint is applied to a fixed number of variables, and indicates which combinations of values are legal. In the MAX CSP problem, the goal is to maximize the number of satisfied constraints. See Section 2 for formal definitions.

The investigation of CSPs has led to deep theorems characterizing the complexity of a CSP based on the type of constraints allowed in the instance [7, 24]. For example, the long-awaited CSP dichotomy theorem [6, 35] provides a criterion separating the NP-complete from the polynomial-time solvable CSPs; the work of Khanna, Sudan, Trevisan, and Williamson characterizes how well the maximization version of a Boolean CSP can be approximated [21] (see Refs. [13, 20] for larger domains; see Refs. [12, 27] for optimal approximation factors); and Cai and Chen [8] present a dichotomy that separates CSPs for which the number of complex-weighted solutions can be counted in polynomial time, from those where the problem is #P-hard.

In this work, we analyze the complexity of constraint satisfaction in an algorithmic regime that is currently far from understood: polynomial-time compression and kernelization [15]. Here, the goal is to analyze how much (in terms of the number of variables n) an instance can be compressed by a polynomial-time algorithm without changing the answer, and to understand how the compressibility depends on the type of available constraints. A *compression* is a polynomial-time algorithm that reduces instances of one problem to equivalent, small instances of a potentially different problem; a *kernelization* compresses to an instance of the same problem (see Section 2.4). A kernelization of small size allows an instance to be stored, manipulated, and solved more efficiently. It is, therefore, of interest to find the smallest possible kernelizations (in terms of the output size). Since every kernelization yields a compression, one can prove lower bounds on the size of kernelizations by establishing lower bounds on compressions.

In recent years, there have been a number of advances in the understanding of compressibility of CSPs [9, 14, 18, 25]. A foundational result by Dell and van Melkebeek [14] states that for $d \geq 3$, CNF-SAT with clauses of size at most d (d -CNF-SAT) parameterized by the number of variables n admits no (polynomial-time) compression of size $O(n^{d-\epsilon})$ for any $\epsilon > 0$, unless $\text{NP} \subseteq \text{co-NP/poly}$ (which is known to imply a collapse of the polynomial hierarchy [34]). As an instance of d -CNF-SAT can trivially be compressed to $O(n^d)$ bits via a bitstring that encodes for each of the $O(n^d)$ possible clauses whether or not it is present in the instance, the d -CNF-SAT problem does not admit any non-trivial compression. The situation is different for the related problem d -NOT-ALL-EQUAL SAT (d -NAE-SAT), which is the variant where a clause is satisfied when its literals do not all evaluate to the same value. Jansen and Pieterse showed [17, 18] that for $d \geq 3$, the d -NAE-SAT problem has a compression of size $O(n^{d-1} \log n)$, but not of size $O(n^{d-1-\epsilon})$ unless $\text{NP} \subseteq \text{co-NP/poly}$. This example shows that the type of constraints affects the compressibility of a CSP.

The notion of a *constraint language* is used to rigorously analyze how the complexity of a CSP depends on the type of constraints. In this work, we will only consider CSPs over the Boolean domain: we work exclusively with Boolean constraints and constraint languages. A constraint

is therefore a function of the form $f: \{0, 1\}^k \rightarrow \{0, 1\}$, where $k \geq 1$ is the *arity* of the constraint, also denoted as $\text{AR}(f)$. A constraint language Γ is a *finite* set of constraints. The input of the corresponding decision problem, denoted $\text{CSP}(\Gamma)$, consists of a set of constraint applications of the form $f(x_{j_1}, \dots, x_{j_{\text{AR}(f)}}) = 1$ over n common variables, where f is some constraint from Γ . The question is whether there is an assignment $\{x_1, \dots, x_n\} \rightarrow \{0, 1\}$ satisfying all the constraint applications.

In this terminology, Chen, Jansen, and Pieterse [9] characterized for all (Boolean) constraint languages Γ consisting of constraints of arity at most three, what the optimal compression size is for $\text{CSP}(\Gamma)$. Lagerkvist and Wahlström [25] gave universal-algebraic conditions on Γ which ensure that $\text{CSP}(\Gamma)$ has a compression of size $O(n \log n)$, and a characterization is known of the constraint languages Γ_{sym} consisting entirely of *symmetric* functions for which $\text{CSP}(\Gamma_{\text{sym}})$ has a compression of near-linear size [9, Section 5]. Hence, there is some understanding of the optimal compressibility of $\text{CSP}(\Gamma)$.

However, when we move from the question of whether *all* constraints can be satisfied to the task of maximizing the number of satisfied constraints (MAX CSP), the situation is much less understood. To the best of our knowledge, no non-trivial compressions are known for any MAX $\text{CSP}(\Gamma)$, and no compression lower bounds are known for MAX $\text{CSP}(\Gamma)$ other than those already implied from $\text{CSP}(\Gamma)$. In this article, we, therefore, analyze the compressibility of MAX $\text{CSP}(\Gamma)$.

Before presenting our results, we briefly summarize the main algorithmic approach for compressing $\text{CSP}(\Gamma)$ and illustrate why it fails completely for MAX CSP. Consider for example 3-NAE-SAT. The number of constraint applications in an n -variable instance of this problem can be reduced to $O(n^2)$ without changing the solution space, which allows it to be encoded in $O(n^2 \log n)$ bits. The *sparsification* to $O(n^2)$ constraint applications is achieved by a linear-algebraic approach. Note that a not-all-equal constraint on variables $(x, y, z) \in \{0, 1\}^3$ is satisfied if and only if $x + y + z - xy - xz - yz - 1 = 0$. Observe that if $p_1(x_1, \dots, x_n) = 0, \dots, p_m(x_1, \dots, x_n) = 0$ are polynomial equalities which are satisfied by an assignment to $x_1, \dots, x_n$, then also $\sum_{i=1}^m \alpha_i \cdot p_i(x_1, \dots, x_n) = 0$ holds for any linear combination as determined by $\alpha_1, \dots, \alpha_m$. To sparsify a 3-NAE-SAT instance with this insight, proceed as follows. Transform each constraint c_i into an equality $p_i(x_1, \dots, x_n) = 0$ for a degree-2 polynomial p_i , substituting $1 - v$ for negated variables $\neg v$ in the constraint. This yields a system of equations of degree-2 polynomials in n variables, which have $O(n^2)$ distinct monomials. The rank of a corresponding vector space is therefore $O(n^2)$, which yields a basis of $O(n^2)$ equalities such that all others can be expressed as their linear combinations. All constraints not corresponding to an element of this basis can be safely omitted from an instance of 3-NAE-SAT, since they will be automatically satisfied by any assignment that satisfies *all* basis constraints. This yields the claimed sparsification of $O(n^2)$ constraints. Note, however, that this approach fails completely for the variant MAX 3-NAE-SAT: if an assignment *does not* satisfy all constraints of the basis, this does not give any satisfaction guarantees on the linearly-dependent constraints. Hence the sparsification approach for $\text{CSP}(\Gamma)$ is not applicable for MAX $\text{CSP}(\Gamma)$.

Our results. We provide a new route to compression for MAX $\text{CSP}(\Gamma)$, and prove that the resulting compressions are *essentially optimal* for all constraint languages Γ , assuming $\text{NP} \not\subseteq \text{co-NP/poly}$. Our results characterize the optimal compressibility of all Boolean MAX CSPs in terms of degrees of characteristic polynomials, and uncover a wide range of MAX $\text{CSP}(\Gamma)$ problems that admit a non-trivial compression. For a Boolean function $f: \{0, 1\}^k \rightarrow \{0, 1\}$, its *characteristic polynomial* is the *unique* k -variate multilinear polynomial $P_f(x)$ over $\mathbb{R}$ that agrees with f on all $x \in \{0, 1\}^k$. The fact that this representation is unique is well-known (cf. [29]). For a constraint language Γ , define $\deg(\Gamma) = \max_{f \in \Gamma} \deg(P_f)$. We prove that $\deg(\Gamma)$ characterizes the compressibility of MAX $\text{CSP}(\Gamma)$.

To state our results precisely, we have to address a feature of the problem that is particular to the maximization variant: repetitions of constraint applications. While such repetitions

are irrelevant in the CSP setting when all constraint applications have to be satisfied, they become relevant when maximizing the number of satisfied constraint applications. The standard approach in the MAX CSP literature is, therefore, to give each constraint application a positive integer weight value [11, 21]. The decision problem MAX CSP(Γ) then takes as input a system of Γ -constraint applications with weights from $\mathbb{N}$, and a threshold value t , and asks whether there is an assignment such that the weight of the satisfied constraint applications is at least t .

Let Γ be a (finite, Boolean) constraint language.¹ Our main positive result is the following.

THEOREM 1.1. *MAX CSP(Γ) parameterized by the number of variables n , with positive integer weights bounded by $n^{O(1)}$, admits a compression of size $O(n^{\deg(\Gamma)} \log n)$.*

In fact, we are even able to reduce any instance of MAX CSP(Γ) to an equivalent instance of the *same* problem, having $O(n^{\deg(\Gamma)})$ weighted constraint applications. We prove matching lower bounds whenever MAX CSP(Γ) is NP-complete. It is known [10, 11, 21] that for inputs with positive integer weights, MAX CSP(Γ) is polynomial-time solvable if Γ is 0-valid, 1-valid, or 2-monotone (see Section 2.1), and NP-complete otherwise.

THEOREM 1.2. *Suppose that Γ is not 0-valid, 1-valid, or 2-monotone. Assuming $NP \not\subseteq co-NP/poly$, MAX CSP(Γ) parameterized by the number of variables n , with positive integer weights bounded by $n^{O(1)}$, does not admit a compression of size $O(n^{\deg(\Gamma)-\varepsilon})$ for any $\varepsilon > 0$.*

Our results uncover an interesting contrast in compressibility between decision CSPs and maximization CSPs. While both involve the analysis of the degrees of polynomials, the type of polynomials which is used differs, leading to differences in compressibility. For example, while d -NAE-SAT has a compression of size $O(n^{d-1} \log n)$ for all $d \geq 3$, the corresponding MAX d -NAE-SAT problem with weights of absolute value $n^{O(1)}$ has a compression of size $O(n^{d-1} \log n)$ for odd $d \geq 3$, but no compression of size $O(n^{d-\varepsilon})$ for even $d \geq 4$. Another example is d -EXACT SAT, where we require exactly one literal in each clause to be true. Whereas d -EXACT SAT admits a compression of size $O(n \log n)$ for every fixed d [9], we show that MAX d -EXACT SAT cannot be compressed to $O(n^{d-\varepsilon})$ bits.

Techniques. On a high level, our results are obtained by combining two ingredients: (1) a characterization of the complexity of a constraint language as $\deg(\Gamma)$ via the degree of the characteristic polynomials, and (2) reductions between different problems MAX CSP(Γ) and MAX CSP(Γ') by implementing constraints of one language by combinations of constraints from the other. While both ingredients have been used in isolation [10, 11, 21, 26, 33], their combination is novel and is the key to understanding compressibility. To comprehend how characteristic polynomials help to compress an instance of MAX CSP(Γ), observe that since the characteristic polynomial gives 1 when a constraint is satisfied and 0 otherwise, the total value of satisfied constraint applications can be written as a weighted sum of applications of characteristic polynomials. If $\deg(\Gamma) = k$, then this weighted sum contains $O(n^k)$ distinct monomials. An instance can, therefore, be compressed by expanding this weighted sum, and storing the coefficient of each monomial. If all weights in the input instance are bounded by $n^{O(1)}$, each coefficient will have value $n^{O(1)}$ and can therefore be encoded in $O(\log n)$ bits.

Our lower bounds are obtained by parameterized reductions between MAX CSPs in which the number of variables does not grow significantly. By a careful analysis of the terms of the characteristic polynomial, we show that if $\deg(\Gamma) = \deg(\Gamma')$, then constraint applications from Γ can effectively be simulated by combinations of constraints from Γ' . Here, we use the framework

¹While some recent work on CSP sparsification allows infinite constraint languages Γ [18], they are not interesting from our perspective as $\deg(\Gamma) = +\infty$.

of *implementations* from an earlier work [21]. Since the characteristic polynomial of d -CNF clauses has degree d , this yields a reduction from d -CNF-SAT to MAX CSP(Γ) for $\deg(\Gamma) = d$ that preserves the asymptotic size of the variable set, therefore transferring the cited lower bound for d -CNF-SAT [14] to MAX CSP(Γ). A similar reduction is also used for our positive results, to turn the monomial-based compression sketched above into a self-reduction, which outputs an instance of the original problem.

Consequences for exponential-time algorithms. The framework we develop for parameterized reductions among MAX CSPs also has consequences for exponential-time algorithms, which we believe to be of independent interest. The MAX 3-SAT HYPOTHESIS [26] states that MAX 3-CNF-SAT with n variables cannot be solved in time $O(2^{(1-\varepsilon)n})$ for any $\varepsilon > 0$ (cf. [1, 5]). Our reductions imply that this hypothesis is *equivalent* to the version where MAX 3-CNF-SAT is replaced by MAX CSP(Γ) for any constraint language Γ with $\deg(\Gamma) = 3$ in which negated literals can be expressed (Section 2.2). In particular, the MAX 3-SAT hypothesis is equivalent to the statement that MAX E3-LIN (formally defined in Section 2.2) cannot be solved in time $O(2^{(1-\varepsilon)n})$. What is more, for any $k \geq 2$, our reductions uncover an equivalence class of NP-hard problems whose optimal exponential-time running times coincide with the one for MAX k -SAT.

Related work. Representations of Boolean functions by characteristic polynomials have been studied frequently in the literature [3, 4, 26, 28, 29, 31, 32], revealing, e.g., a relation between the degree of the representation and the decision tree complexity [29]. Williams [33] combined this approach with the split-and-list technique to give accelerated exponential-time algorithms for MAX 2-SAT and MAX CSP(Γ) for $\deg(\Gamma) = 2$. In recent work, Lincoln, Williams, and Vassilevska Williams [26] give an exponential-time split-and-list reduction from MAX CSP(Γ) for $\deg(\Gamma) = k$ to detecting an ℓ -hyperclique in a k -uniform hypergraph, for $\ell > k$, in support of the (k, ℓ) -HYPERCLIQUE HYPOTHESIS, which states that detecting such a hyperclique in an n -vertex input requires time $n^{\ell-o(1)}$ on a Word-RAM with $O(\log n)$ -bit words. If this hypothesis fails for some k and ℓ , their reduction implies that each MAX CSP(Γ) problem with $\deg(\Gamma) = k$ can be solved in time $O(2^{(1-\varepsilon)n})$ for some $\varepsilon > 0$. Their reductions run in exponential time and are very different from ours.

Alon et al. [2] used a different representation of Boolean functions as polynomials in the work on MAX r -SAT parameterized above the guarantee. Here, the goal is to find an assignment satisfying at least $((2^r - 1)m + k)/2^r$ clauses, where m is the total number of clauses and k is the parameter. They have shown that the problem is FPT and admits a polynomial kernel.

Following the publication of the extended abstract of this work, a superset of the authors [19] has used the framework introduced here to investigate the compressibility of constraint satisfaction problems in the regime of exponentially-large weights, as opposed to the weights of absolute value $n^{O(1)}$ used here. The distinction is important and increases the exponent of the optimal compression size by one in several cases.

Organization. We begin with Section 2, containing the necessary definitions and properties of CSPs, including the implementation framework. In Section 3, we explain the idea of representing constraints by polynomials and provide an algebraic background for our reductions. It is followed by Section 4, where the notion of a reduction between constraint systems is formalized, and the main reductions are presented. It serves as a toolbox for proving the main results for compression (Section 5) and exponential-time algorithms (Section 6).

2 PRELIMINARIES

For a set S and integer $d \geq 0$, let $\binom{S}{d}$ be the set of all size- d subsets of S . We use $[n]$ as a shorthand for $\{1, \dots, n\}$. A k -ary constraint is a function $f: \{0, 1\}^k \rightarrow \{0, 1\}$. We refer to k as the arity of f ,

denoted $\text{AR}(f)$. We always assume that the domain is Boolean. A constraint f is satisfied by an input $s \in \{0, 1\}^k$ if $f(s) = 1$. A constraint language (sometimes called constraint family) Γ is a finite collection of constraints $\{f_1, f_2, \dots, f_\ell\}$, potentially with different arities. A *constraint application*, of a k -ary constraint f to a set of n Boolean variables, is a triple $\langle f, (i_1, i_2, \dots, i_k), w \rangle$, where the indices $i_j \in [n]$ select k of the n Boolean variables to whom the constraint is applied, and w is a weight, described formally below. The variables can be repeated in a single application.

Definition 2.1. A *constraint system* is a pair $\text{CS}(\Gamma, \mathbb{W})$, where Γ is a constraint language and $\mathbb{W}$ (for *weight range*) is either $\mathbb{Z}$ or $\mathbb{N}$. An *instance* (or *formula*) of $\text{CS}(\Gamma, \mathbb{W})$ is a set of constraint applications from Γ over a common set of variables, each application having a weight from $\mathbb{W}$.

We denote the number of constraint applications in a formula Φ by $|\Phi|$ and the sum of absolute values of all weights in Φ by $\|\Phi\|$. For an assignment x , that is, a mapping from the set of variables to $\{0, 1\}$, the integer $\Phi(x)$ is the sum of weights of the constraint applications satisfied by x .

In the decision problem $\text{MAX CSP}(\Gamma, \mathbb{W}, c)$, we are given a formula Φ from $\text{CS}(\Gamma, \mathbb{W})$ over n variables such that $\|\Phi\| \leq n^c$, together with the integer t , and we ask if there is an assignment x such that $\Phi(x) \geq t$. We specify the constant c to be explicit about the specific decision problems for which we show hardness results. This is needed because the complexity of MAX CSP can differ depending on how large the weights are, corresponding to the distinction between weak and strong NP-completeness. (The formal definitions of parameterized problems and compression are given in Section 2.4.) When it does not lead to confusion, e.g., when some property holds for all c , we refer to the corresponding family of problems shortly as $\text{MAX CSP}(\Gamma, \mathbb{W})$. Whenever we use the O -notation, we do it with respect to a fixed problem, that is, we treat Γ and c as constants. The most commonly studied case is expressed by $\mathbb{W} = \mathbb{N}$ [13, 21, 27], where the weights can be interpreted as repetitions of constraint applications. It is important to make this distinction because it can be the case that $\text{MAX CSP}(\Gamma, \mathbb{N})$ is polynomially solvable whereas $\text{MAX CSP}(\Gamma, \mathbb{Z})$ is NP-hard [20]. Although our main reduction framework works for $\mathbb{W} = \mathbb{Z}$, we are able to transfer the compression lower bounds to the case $\mathbb{W} = \mathbb{N}$ as long as $\text{MAX CSP}(\Gamma, \mathbb{N})$ is NP-hard.

Another decision problem that is related to constraint systems is $\text{EXACT CSP}(\Gamma, \mathbb{W})$, where we ask whether there is an assignment for which the satisfied weights sum up exactly to a given integer [26, 33]. Even though we focus on the maximization variant, we formulate our reductions so that they could be employed for other problems [19] over constraint systems or larger weight domains.

2.1 Types of Constraints

We start by formally defining the most important constraints and constraint properties. They allow us to formulate the dichotomy theorem for MAX CSP . We use the Boolean notation for negation, i.e., $\neg x = 1 - x$ for $x \in \{0, 1\}$.

- A constraint is *trivial* if it is either always 1 or always 0 regardless of the arguments.
- The unary constraints T and F are given by $T(x) = x$ and $F(x) = \neg x$.
- OR_k and AND_k are k -ary constraints, such that $\text{OR}_k(x_1, \dots, x_k) = \bigvee_{i=1}^k x_i$ and analogously $\text{AND}_k(x_1, \dots, x_k) = \bigwedge_{i=1}^k x_i$. The Not-All-Equal constraint is defined as $\text{NAE}_k(x_1, \dots, x_k) = \text{OR}_k(x_1, \dots, x_k) \wedge \text{OR}_k(\neg x_1, \dots, \neg x_k)$.
- XOR_k is a k -ary constraint defined as $\text{XOR}_k(x_1, \dots, x_k) = x_1 + \dots + x_k \bmod 2$. We abbreviate $\text{XOR} = \text{XOR}_2$.
- A constraint f is *0-valid* (resp. *1-valid*) if $f(0, 0, \dots, 0) = 1$ (resp. $f(1, 1, \dots, 1) = 1$).
- A constraint f is *2-monotone* if $f(x_1, x_2, \dots, x_k) = (x_{i_1} \wedge x_{i_2} \wedge \dots \wedge x_{i_p}) \vee (\neg x_{j_1} \wedge \neg x_{j_2} \wedge \dots \wedge \neg x_{j_q})$, for some $p, q \geq 0$, $(p, q) \neq (0, 0)$, i.e., f is equivalent to a DNF-formula with at

most two terms: one containing only positive literals and the other containing only negative literals.

- A constraint f is *C-closed* (complementation-closed) if for every assignment $x \in \{0, 1\}^{\text{Ar}(f)}$, $f(x) = f(\widehat{x})$, where $\widehat{x}$ stands for the bit-wise complement of x .
- A constraint f is *symmetric* if for any two assignments $x_1, x_2 \in \{0, 1\}^{\text{Ar}(f)}$ having the same number of ones, it holds that $f(x_1) = f(x_2)$.

A constraint language Γ is called 0-valid, 1-valid, 2-monotone, C-closed, or symmetric, if all non-trivial constraints in Γ satisfy the respective property. We call Γ *non-trivial* if it contains at least one non-trivial constraint. This regime is convenient for formulating the fundamental dichotomy theorem for Boolean MAX CSP. For our purposes, it is only important that APX-hardness entails NP-hardness.

THEOREM 2.2 ([21, THEOREM 2.11], CF. [10]). *MAX CSP($\Gamma, \mathbb{N}$) is solvable in polynomial time if Γ is either 0-valid, 1-valid, or 2-monotone. Otherwise, the problem is APX-hard.*

2.2 Closures of Constraint Languages

In some CSPs, we are allowed to write constraint applications containing constants or negations of variables, which makes them more convenient to process. We formalize these properties with the notion of a *language closure*.

Definition 2.3. Let f be a k -ary constraint and let g be a d -ary constraint. We say that g is *expressible by f with constants* if the identity $g(x_1, x_2, \dots, x_d) = f(\xi_1, \xi_2, \dots, \xi_k)$ holds for a tuple $(\xi_1, \xi_2, \dots, \xi_k)$, where each ξ_j is either a variable x_i for some $i \in [d]$ or one of the constants 0, 1.

We say that g is *expressible by f with literals* if such an identity holds for a tuple $(\xi_1, \xi_2, \dots, \xi_k)$, where each ξ_j is a literal: either a variable x_i or its negation $\neg x_i$ for $i \in [d]$.

For a constraint language Γ , we introduce the following closures:

- the language $\Gamma^{T,F}$ contains all functions expressible by $f \in \Gamma$ with constants,
- the language Γ^{LIT} contains all functions expressible by $f \in \Gamma$ with literals,
- the language Γ^{NEG} is the *negation-wise closure* of Γ , i.e., $\Gamma^{NEG} = \bigcup_{f \in \Gamma} \{f, \neg f\}$.

It is easy to see that the closures satisfy $(\Gamma^{T,F})^{T,F} = \Gamma^{T,F}$, $(\Gamma^{LIT})^{LIT} = \Gamma^{LIT}$, $(\Gamma^{NEG})^{NEG} = \Gamma^{NEG}$. We will be particularly interested in those constraint languages in which negated literals can be expressed, as in, e.g., d -CNF-SAT or d -NAE-SAT. These are the languages that satisfy $\Gamma = \Gamma^{LIT}$. Below we present examples on how to express important CSPs using our definitions.

- d -CNF-SAT = CSP($\Gamma_{d\text{-SAT}}$) for $\Gamma_{d\text{-SAT}} = \{\text{OR}_d\}^{LIT}$,
- d -NAE-SAT = CSP($\{\text{NAE}_d\}^{LIT}$),
- MAX Ed-LIN = MAX CSP($\{\text{XOR}_d\}^{NEG}, \mathbb{N}$),
- MAX CUT = MAX CSP($\{\text{XOR}\}, \mathbb{N}$),
- MAX DiCUT = MAX CSP($\{f\}, \mathbb{N}$) for $f(x_1, x_2) = x_1 \wedge \neg x_2$.

2.3 Constraint Implementations

We describe the technique behind Theorem 2.2 from [21]. The idea is to *implement* a constraint f by a collection of other constraints, so that satisfying f is equivalent to maximizing the number of satisfied constraints in that collection. It allows us to express formulas from MAX CSP($\Gamma_1, \mathbb{N}$) by those from MAX CSP($\Gamma_2, \mathbb{N}$), as long as constraints in Γ_1 can be implemented by those in Γ_2 .

The caveat is that each implementation may introduce new auxiliary variables, whereas for our purposes, we need reductions that increase the number of variables only by a multiplicative constant. Therefore, the reductions by Khanna et al. [21] do not transfer compressibility bounds;

we will use the implementations in a different way. On the other hand, our reductions do not preserve approximation factors.

Definition 2.4 ([21, Definition 3.1]). A collection of unit-weighted constraint applications $C_1, C_2, \dots, C_m$ over a set of variables $x = \{x_1, x_2, \dots, x_p\}$ called primary variables and $y = \{y_1, y_2, \dots, y_q\}$ called auxiliary variables, is an α -implementation of a constraint $f(x)$ for a positive integer α if the following conditions hold.

- (1) Any assignment to x and y satisfies at most α constraint applications from $C_1, C_2, \dots, C_m$.
- (2) For all assignments to x such that $f(x) = 1$, there exists an assignment to y such that exactly α constraint applications are satisfied by x and y .
- (3) For all assignments to x and y such that $f(x) = 0$, at most $\alpha - 1$ constraint applications are satisfied by x and y .

An α -implementation is called *strict* if for every assignment of the primary variables x such that $f(x) = 0$, there exists an assignment of the auxiliary variables y such that exactly $\alpha - 1$ constraint applications are satisfied.

We say that a constraint language Γ implements a constraint f if there exists an α -implementation of f using constraints of Γ for some constant α . We use $\Gamma \Rightarrow f$ to denote that Γ implements f and $\Gamma \xRightarrow{s} f$ when Γ strictly implements f . The above notation is also extended to allow the target to be a family of constraints. The following lemma encapsulates a toolbox of implementations that we will rely on.

LEMMA 2.5. *If Γ is a non-trivial constraint language such that*

- (1) Γ is C -closed and not 0-valid (or equivalently not 1-valid), then $\Gamma \Rightarrow \text{XOR}$,
- (2) Γ is neither 0-valid, 1-valid, nor C -closed, then $\Gamma \Rightarrow \{T, F\}$,
- (3) Γ is neither 0-valid, 1-valid, nor 2-monotone, then $\Gamma \Rightarrow \text{XOR}$.

In order to prove it, we need to refer to several statements from Ref. [21], beginning from the transitivity of strict implementations. Then, we restate three lemmas that imply points (1, 2) directly, and point (3) is obtained via transitivity.

LEMMA 2.6 ([21, LEMMA 3.5]). *If $\Gamma_1 \xRightarrow{s} \Gamma_2$ and $\Gamma_2 \xRightarrow{s} \Gamma_3$, then $\Gamma_1 \xRightarrow{s} \Gamma_3$.*

LEMMA 2.7 ([21, LEMMA 4.5]). *Let f be a non-trivial constraint which is C -closed and is not 0-valid (or equivalently not 1-valid). Then $\{f\} \xRightarrow{s} \text{XOR}$.*

LEMMA 2.8 ([21, LEMMA 4.6]). *Let f_0, f_1 , and g be non-trivial constraints, possibly identical, which are not 0-valid, not 1-valid, and not C -closed, respectively. Then $\{f_0, f_1, g\} \xRightarrow{s} \{T, F\}$.*

LEMMA 2.9 ([21, LEMMA 4.11]). *Let f be a constraint which is not 2-monotone. Then $\{f, T, F\} \xRightarrow{s} \text{XOR}$.*

PROOF OF LEMMA 2.5. Claims (1, 2) follow from Lemmas 2.7 and 2.8, respectively. To see claim (3), first observe that if Γ is C -closed, then we can again use Lemma 2.7. Otherwise, by Lemma 2.8 we have $\Gamma \xRightarrow{s} \{T, F\}$. Next, we take advantage of transitivity (Lemma 2.6) and implement XOR with Lemma 2.9. $\square$

2.4 Parameterized Complexity

A *parameterized problem* is a decision problem in which every input has an associated positive integer *parameter* that captures its complexity in some well-defined way. In our study of CSPs, we

use the number of variables as the parameter, but other choices have been considered [16, 22, 23]. For a parameterized problem $A \subseteq \Sigma^* \times \mathbb{N}$, a decision problem $B \subseteq \Sigma^*$, and a function $f: \mathbb{N} \rightarrow \mathbb{N}$, a *compression* of A into B of size f is an algorithm that, on input $(x, k) \in \Sigma^* \times \mathbb{N}$, takes time polynomial in $|x| + k$ and outputs an instance $y \in \Sigma^*$ such that $(x, k) \in A$ if and only if $y \in B$, and such that $|y| \leq f(k)$. A *kernelization* algorithm of size f for problem A reduces any instance (x, k) to an $f(k)$ -sized equivalent instance of the *same problem* in polynomial time.

3 CHARACTERISTIC POLYNOMIALS

In this section, we provide the technique necessary for expressing one constraint system by another without introducing too many auxiliary variables. This insight is based on interpreting constraints as multilinear polynomials.

Definition 3.1. For a k -ary constraint $f: \{0, 1\}^k \rightarrow \{0, 1\}$ its *characteristic polynomial* P_f is the unique k -ary multilinear polynomial over $\mathbb{R}$ satisfying $f(x) = P_f(x)$ for any $x \in \{0, 1\}^k$.

It is easy to construct P_f . First define $P_s(x_1, x_2, \dots, x_k) = \prod_{i=1}^k R_i^s(x_i)$ for a vector $s \in \{0, 1\}^k$, where $R_i^s(x) = x$ if $s_i = 1$ and $R_i^s(x) = 1 - x$ otherwise. Formally, P_s is given by the sequence of coefficients obtained by expanding all parentheses; they are all integers. It holds that $P_s(s) = 1$, while $P_s(x) = 0$ for any $x \neq s$. For a constraint f its characteristic polynomial is given as $P_f(x_1, x_2, \dots, x_k) = \sum_{s: f(s)=1} P_s(x_1, x_2, \dots, x_k)$. It is known that no other multilinear polynomial can take identical values on $\{0, 1\}^k$ [29, 33]. This also means we can interchangeably analyze polynomials as formal objects and as functions on $\{0, 1\}^k$.

OBSERVATION 1. For any Boolean function f , the coefficients of the characteristic polynomial P_f are integers.

Let $\deg(f) = \deg(P_f)$ and $\deg(\Gamma) = \max_{f \in \Gamma} \deg(f)$. For a k -ary constraint f we refer to the coefficient of the unique k -ary monomial in P_f as the *leading coefficient*. The leading coefficient is non-zero if and only if $\deg(P_f) = k$. If g is expressible by f with literals, then we can obtain P_g from P_f by replacing each literal with negation $\neg x_i$ by $1 - x_i$ and expanding the parenthesis within monomials. If g is expressible by f with constants, then we just substitute 0 or 1 for particular variables and remove monomials containing 0. These transformations imply $\deg(\Gamma^{T,F}) = \deg(\Gamma^{LIT}) = \deg(\Gamma^{NEG}) = \deg(\Gamma)$.

Example 3.2. Consider MAX 3-NAE-SAT. The function $\text{NAE}_3(x_1, x_2, x_3)$ has the degree-2 characteristic polynomial $x_1 + x_2 + x_3 - x_1x_2 - x_1x_3 - x_2x_3$, which allows us to construct a compression for MAX 3-NAE-SAT of size $O(n^2 \log n)$ by summing coefficients at all $O(n^2)$ monomials. On the other hand, $\text{OR}_2(x_1, x_2) = x_1 + x_2 - x_1x_2 = \text{NAE}_3(x_1, x_2, 0)$, which indicates that solving MAX 3-NAE-SAT should not be easier than MAX 2-CNF-SAT. We will formalize these arguments in the next section.

We show that the set of characteristic polynomials of all constraints expressible by f with constants spans the linear space of multilinear polynomials over $\mathbb{Q}$ with degrees at most $\deg(f)$. We first prove in Lemma 3.3 that this set contains polynomials of all degrees up to $\deg(f)$ and then use them in Lemma 3.4 to express a basis of the linear space.

LEMMA 3.3. *Let f be a k -ary constraint. For any $1 \leq d \leq \deg(f)$ there exists a d -ary constraint g expressible by f with constants, such that its characteristic polynomial P_g has degree exactly d , i.e., its leading coefficient is non-zero.*

PROOF. The degree of d -ary P_g can be at most d so we just need to show that the leading coefficient is non-zero. We proceed by induction over $i = \deg(f) - d$. Suppose that $d = \deg(f) = \deg(P_f)$ and consider indices $j_1, \dots, j_d$ for which a monomial $\prod_{i=1}^d x_{j_i}$ has a non-zero coefficient α in P_f .

We define $g_0(x_1, x_2, \dots, x_d) = f(\xi_1, \xi_2, \dots, \xi_k)$, where $\xi_{j_i} = x_i$ and other ξ_j are set to 0. It is easy to verify that the leading coefficient of P_{g_0} is α , which proves the induction basis for $i = 0$.

Suppose now the induction hypothesis holds for $i - 1$, that is, $g_{d+1}(x_1, x_2, \dots, x_{d+1})$ is expressible by f with constants and $\deg(P_{g_{d+1}}) = d + 1$. First consider the case where $P_{g_{d+1}}$ has a non-zero coefficient at some monomial of degree d . Let j be the unique index for which x_j does not occur in this monomial. Then $g_d(x_1, x_2, \dots, x_d) = g_{d+1}(x_1, \dots, x_{j-1}, 0, x_j, \dots, x_d)$ is expressible by f with constants and has degree d .

Finally, suppose that $P_{g_{d+1}}$ has zero coefficients at all monomials of degree d . We define $g_d(x_1, x_2, \dots, x_d) = g_{d+1}(x_1, x_2, \dots, x_d, 1)$. Since the monomial $\prod_{i=1}^d x_i$ does not appear in $P_{g_{d+1}}$, the leading coefficient in P_{g_d} is the same as in $P_{g_{d+1}}$. $\square$

LEMMA 3.4. *Let f be a non-trivial constraint and P be a multilinear polynomial over $\mathbb{Q}$ on ℓ variables of degree $0 \leq d \leq \deg(f)$. There exists a sequence of constraint applications $\langle f_i, (j_i^1, \dots, j_i^{\text{AR}(f_i)}), \alpha_i \rangle_{i=1}^M$ on ℓ variables, where each constraint f_i is expressible by f with constants and $\alpha_i \in \mathbb{Q}$, such that the following polynomial identity holds.*

$$P(x_1, \dots, x_\ell) = \sum_{i=1}^M \alpha_i \cdot P_{f_i}(x_{j_i^1}, \dots, x_{j_i^{\text{AR}(f_i)}}).$$

Before proving this claim, let us demonstrate, it on a less obvious example than the one with NAE₃ and OR₂.

Example 3.5. Let $P(x_1, x_2, x_3)$ be the characteristic polynomial of the constraint OR₃($x_1, x_2, \neg x_3$), derived using the method described below Definition 3.1:

$$\begin{aligned} P(x_1, x_2, x_3) &= x_1 x_2 x_3 + (1 - x_1) x_2 x_3 + x_1 (1 - x_2) x_3 + x_1 x_2 (1 - x_3) \\ &\quad + (1 - x_1) x_2 (1 - x_3) + x_1 (1 - x_2) (1 - x_3) + (1 - x_1) (1 - x_2) (1 - x_3) \\ &= 1 - x_3 + x_1 x_3 + x_2 x_3 - x_1 x_2 x_3. \end{aligned}$$

We will represent it with characteristic polynomials from $\{\text{EX}_3\}^{T,F}$, where $\text{EX}_3(x_1, x_2, x_3) = 1$ if and only if exactly one variable is 1. Its characteristic polynomial Q is given as:

$$\begin{aligned} Q(x_1, x_2, x_3) &= x_1 (1 - x_2) (1 - x_3) + (1 - x_1) x_2 (1 - x_3) + (1 - x_1) (1 - x_2) x_3 \\ &= x_1 + x_2 + x_3 - 2x_1 x_2 - 2x_1 x_3 - 2x_2 x_3 + 3x_1 x_2 x_3. \end{aligned}$$

We can express P as the following linear combination where, e.g., $Q(x_1, x_2, 0)$ is the characteristic polynomial for $\text{EX}_3(x_1, x_2, 0)$, which is a binary constraint expressible by EX_3 with constants:

$$\begin{aligned} P(x_1, x_2, x_3) &= -\frac{1}{3}Q(x_1, x_2, x_3) + \frac{1}{3}Q(x_1, x_2, 0) - \frac{1}{6}Q(x_1, x_3, 0) - \frac{1}{6}Q(x_2, x_3, 0) \\ &\quad + \frac{1}{6}Q(x_1, 0, 0) + \frac{1}{6}Q(x_2, 0, 0) - \frac{1}{3}Q(x_3, 0, 0) + Q(1, 0, 0). \end{aligned}$$

PROOF OF LEMMA 3.4. We proceed by induction over the degree of P . Since f is non-trivial, it admits a satisfying assignment s_T . If P is constant, then $P(x) = \alpha \cdot f(s_T)$ for some $\alpha \in \mathbb{Q}$. Suppose now $d = \deg(P) \geq 1$. For each $S \in \binom{[\ell]}{d}$ let α_S denote the (potentially zero) coefficient in P at the monomial $\prod_{i \in S} x_i$. By Lemma 3.3, there is a d -ary constraint f_d , which is expressible by f with constants and $\deg(P_{f_d}) = d$. Let β_d denote the leading coefficient of P_{f_d} . The polynomial

$$P'(x_1, \dots, x_\ell) = P(x_1, \dots, x_\ell) - \sum_{\substack{\{i_1, \dots, i_d\} = S \in \binom{[\ell]}{d} \\ i_1 < \dots < i_d}} \frac{\alpha_S}{\beta_d} \cdot P_{f_d}(x_{i_1}, \dots, x_{i_d})$$

has no monomials of degree d , since each term in the sum subtracts exactly one of them. The polynomial P' has degree at most $d-1$, so we can apply the induction hypothesis to it and represent P' as a linear combination of characteristic polynomials of constraints from $\{f\}^{T,F}$. We obtain the claim by adding these polynomials to the sum above. $\square$

Since f_i and P_{f_i} coincide as functions on $\{0, 1\}^{\text{AR}(f_i)}$, Lemma 3.4 allows us to represent any constraint of degree at most $\deg(f)$ as a linear combination of constraints from $\{f\}^{T,F}$.

PROPOSITION 3.6. *Let g, f be constraints, such that f is non-trivial and $\deg(g) \leq \deg(f)$. There exists a sequence of constraint applications $\langle f_i, (j_i^1, \dots, j_i^{\text{AR}(f_i)}), \alpha_i \rangle_{i=1}^M$ on $\text{AR}(g)$ variables, where each constraint f_i is expressible by f with constants, and $\alpha_i \in \mathbb{Q}$, such that $g(x_1, \dots, x_{\text{AR}(g)}) = \sum_{i=1}^M \alpha_i \cdot f_i(x_{j_i^1}, \dots, x_{j_i^{\text{AR}(f_i)}})$ for all Boolean assignments.*

This resembles the idea of implementation, where we additionally equip constraints with (potentially negative) rational weights, but does not require introducing new variables.

4 REDUCTIONS BETWEEN CONSTRAINT SYSTEMS

We first formalize our notion of reduction. The objects we work with are constraint systems and the reductions between them imply analogous relations between the associated decision problems. The reduction is crafted in such a way that it preserves the numbers of variables and constraints up to a constant factor, and the total weight up to a polynomial factor.

Definition 4.1. A linear transformation from a constraint system $CS(\Gamma_1, \mathbb{W}_1)$ to another constraint system $CS(\Gamma_2, \mathbb{W}_2)$ is a polynomial-time procedure that given a formula Φ_1 of $CS(\Gamma_1, \mathbb{W}_1)$ over n_1 variables and integer t_1 , returns a formula $\Phi_2 \in CS(\Gamma_2, \mathbb{W}_2)$ over n_2 variables and integer t_2 , so that the following conditions hold:

- (1) $n_2 = O(n_1)$,
- (2) $|\Phi_2| = O(|\Phi_1| + n_1)$,
- (3) $\|\Phi_2\| \leq \|\Phi_1\| \cdot n_1^{O(1)}$,
- (4) $\exists_x \Phi_1(x) = t_1 \iff \exists_y \Phi_2(y) = t_2$,
- (5) $\exists_x \Phi_1(x) \geq t_1 \iff \exists_y \Phi_2(y) \geq t_2$.

If there exists a linear transformation from $CS(\Gamma_1, \mathbb{W}_1)$ to $CS(\Gamma_2, \mathbb{W}_2)$, we write $CS(\Gamma_1, \mathbb{W}_1) \leq_{LIN} CS(\Gamma_2, \mathbb{W}_2)$. If (1) can be replaced with the stronger condition $n_2 = n_1 + O(1)$, we call the transformation *additive* and write $CS(\Gamma_1, \mathbb{W}_1) \leq_{ADD} CS(\Gamma_2, \mathbb{W}_2)$.

Before moving forward, let us explain the importance of linear and additive transformations. We formulate two propositions, which follow from the properties in Definition 4.1. For completeness, we provide an explicit proof for the second one.

PROPOSITION 4.2. *If $CS(\Gamma_1, \mathbb{W}_1) \leq_{LIN} CS(\Gamma_2, \mathbb{W}_2)$ and $\text{MAX CSP}(\Gamma_2, \mathbb{W}_2, c)$ admits a compression of size $O(n^d)$, then $\text{MAX CSP}(\Gamma_1, \mathbb{W}_1, c - O(1))$ also admits a compression of size $O(n^d)$.*

PROPOSITION 4.3. *If $CS(\Gamma_1, \mathbb{W}_1) \leq_{ADD} CS(\Gamma_2, \mathbb{W}_2)$ and $\text{MAX CSP}(\Gamma_2, \mathbb{W}_2, c)$ admits an algorithm with running time $T(n)$, then $\text{MAX CSP}(\Gamma_1, \mathbb{W}_1, c - O(1))$ admits an algorithm with running time $T(n + O(1)) + n^{O(1)}$.*

PROOF. Assume that $\text{MAX CSP}(\Gamma_2, \mathbb{W}_2, c)$ admits an algorithm with running time $T(n)$ for some c , so that the time needed to solve an instance only depends on the number of variables n , independent of the number of clauses and the weights in the instance. Note that this is reasonable since the constraint languages we consider are finite, so that the number of different constraint applications is bounded polynomially in the number of variables n , while also the absolute values

of the weights are bounded by n^c based on the definition of $\text{MAX CSP}(\Gamma_2, \mathbb{W}_2, c)$. Suppose there is an additive transformation from formulas Φ_1 of $\text{CS}(\Gamma_1, \mathbb{W}_1)$ on n_1 variables to formulas Φ_2 of $\text{CS}(\Gamma_2, \mathbb{W}_2)$ on $n_2 \leq n_1 + O(1)$ variables. Let $\alpha > 0$ such that the transformation satisfies $\|\Phi_2\| \leq \|\Phi_1\| \cdot n_1^\alpha$, which exists by definition.

Consider an instance (Φ_1, t_1) of $\text{MAX CSP}(\Gamma_1, \mathbb{W}_1, c - \alpha)$ on n_1 variables, which implies $\|\Phi_1\| \leq n_1^{c-\alpha}$. Under the stated conditions, the instance can be solved as follows. We start by applying the additive transformation to the pair Φ_1 and t_1 to obtain a formula Φ_2 on $n_2 \leq n_1 + O(1)$ variables and integer t_2 . We may assume $n_2 \geq n_1$ by inserting dummy variables if needed. The definition guarantees that there is an assignment x_1 with $\Phi_1(x_1) \geq t_1$ if and only if there is an assignment x_2 with $\Phi_2(x_2) \geq t_2$, so the corresponding instances of MAX CSP are equivalent.

The pair consisting of formula Φ_2 and target t_2 forms a valid instance of $\text{MAX CSP}(\Gamma_2, \mathbb{W}_2, c)$, since $\|\Phi_2\| \leq \|\Phi_1\| \cdot n_1^\alpha \leq (n_1)^{c-\alpha} \cdot n_1^\alpha = n_1^c \leq n_2^c$. Hence, we may run an algorithm for $\text{MAX CSP}(\Gamma_2, \mathbb{W}_2, c)$ on (Φ_2, t_2) to resolve the instance, which is equivalent to the original. Since $n_2 \leq n_1 + O(1)$, this takes time $T(n_1 + O(1))$. As the transformation itself can also be computed in polynomial time, the overall running time is bounded by $T(n_1 + O(1)) + n_1^{O(1)}$. $\square$

In particular, additive transformations preserve running times of the form $2^{(1-\varepsilon)n} n^{O(1)}$.

LEMMA 4.4. *Linear transformations (resp. additive transformations) are transitive.*

PROOF. Suppose $\text{CS}(\Gamma_1, \mathbb{W}_1) \leq_{\text{LIN}} \text{CS}(\Gamma_2, \mathbb{W}_2) \leq_{\text{LIN}} \text{CS}(\Gamma_3, \mathbb{W}_3)$ and we want to show that $\text{CS}(\Gamma_1, \mathbb{W}_1) \leq_{\text{LIN}} \text{CS}(\Gamma_3, \mathbb{W}_3)$. Since $n_2 = O(n_1)$ and $n_3 = O(n_2)$, then $n_3 = O(n_1)$. To ensure properties (2, 3) are preserved, note that $|\Phi_3| = O(|\Phi_2| + n_2) = O(|\Phi_1| + n_1)$ and $\|\Phi_3\| \leq \|\Phi_1\| \cdot n_1^{O(1)} \cdot n_2^{O(1)} = \|\Phi_1\| \cdot n_1^{O(1)}$. The properties (4, 5) are equivalences and hence transitive. Moreover, if we replace relation $\leq_{\text{LIN}}$ with $\leq_{\text{ADD}}$, then we have that $n_3 = n_1 + O(1)$ and the other properties follow since an additive transformation is always a linear one. $\square$

We continue with two simple additive transformations, which will allow us to use negations of constraints as an alternative to setting negative weights.

LEMMA 4.5. *For every constraint language Γ we have*

- (1) $\text{CS}(\Gamma^{\text{NEG}}, \mathbb{Z}) \leq_{\text{ADD}} \text{CS}(\Gamma, \mathbb{Z})$,
- (2) $\text{CS}(\Gamma^{\text{NEG}}, \mathbb{Z}) \leq_{\text{ADD}} \text{CS}(\Gamma^{\text{NEG}}, \mathbb{N})$.

PROOF. Suppose, we are given $\Phi_1 \in \text{CS}(\Gamma^{\text{NEG}}, \mathbb{Z})$ and integer t . Observe that $(\neg f)(x) = 1 - f(x)$. For each negated constraint $\neg f$ we can thus remove each constraint application $\langle \neg f, (i_1, i_2, \dots, i_k), w \rangle$, replace it with $\langle f, (i_1, i_2, \dots, i_k), -w \rangle$, and subtract w from the target weight t . We iterate this operation until all applied constraints belong to Γ . We obtain a new formula Φ_2 of $\text{CS}(\Gamma, \mathbb{Z})$ over the same set of variables and a new threshold t' , so that $\Phi_2(x) = t' \iff \Phi_1(x) = t$ (and the same holds with “=” replaced by “ $\geq$ ”). This proves (1).

For claim (2), consider a formula Φ_1 of $\text{CS}(\Gamma^{\text{NEG}}, \mathbb{Z})$. As before, we have $f(x) = 1 - (\neg f)(x)$ and we can replace each constraint application of f having a negative weight by $(\neg f)(x)$ with the opposite weight, and subtract the weight from t . By erasing all negative weights, we obtain a formula Φ_2 of $\text{CS}(\Gamma^{\text{NEG}}, \mathbb{N})$ over the same set of variables and new threshold t' , which satisfies all conditions of Definition 4.1. $\square$

The rest of this section contains several lemmas that form a chain of reductions from $\text{CS}(\Gamma_1, \mathbb{Z})$ to $\text{CS}(\Gamma_2, \mathbb{N})$, which is valid as long as $\deg(\Gamma_1) \leq \deg(\Gamma_2)$ and $\text{MAX CSP}(\Gamma_2, \mathbb{N})$ is NP-hard. First, we translate Proposition 3.6 into the language of additive transformations. In the proof, we justify that one can replace rational coefficients with integer ones.

LEMMA 4.6. *Let Γ_1, Γ_2 be non-trivial constraint languages satisfying $\deg(\Gamma_1) \leq \deg(\Gamma_2)$. Then $CS(\Gamma_1, \mathbb{Z}) \leq_{ADD} CS(\Gamma_2^{T,F}, \mathbb{Z})$.*

PROOF. Let $f \in \Gamma_2$ be a constraint of degree $\deg(\Gamma_2)$. By Proposition 3.6, we can represent each $g \in \Gamma_1$ as a linear combination of constraint applications $\langle f_i, (j_i^1, \dots, j_i^{\text{AR}(f_i)}), \alpha_i \rangle$, where $\alpha_i \in \mathbb{Q}$, $(j_1, \dots, j_{\text{AR}(f_i)}) \in \binom{[\text{AR}(g)]}{\text{AR}(f_i)}$, and functions f_i are expressible by f with constants.

Since Γ_1 is finite, there is a common denominator of all rational numbers occurring in these linear combinations. Therefore, we can find a single positive integer β , so that for each $g \in \Gamma_1$, the function $\beta \cdot g$ can be represented as a combination of constraints from $\{f\}^{T,F}$ with integer coefficients.

Given a formula $\Phi_1 \in CS(\Gamma_1, \mathbb{Z})$, we replace each constraint application $\langle g, (j_1, \dots, j_k), w \rangle$ with a set of constraints applications from $\{f\}^{T,F}$ representing $\beta \cdot g$ and multiply their integer weights by w . We define $t_2 = \beta \cdot t_1$. We obtain a formula $\Phi_2 \in CS(\Gamma_2^{T,F}, \mathbb{Z})$ over the same set of variables, such that $\Phi_2(x) = \beta \cdot \Phi_1(x)$. This implies the properties (4, 5) from Definition 4.1. We have not increased the number of variables, so the transformation is additive, and $|\Phi_1|, ||\Phi_1||$ have been increased by a constant factor depending only on Γ_1 and Γ_2 . $\square$

Now, we formalize an intuitive notation for combining formulas. Consider two formulas Φ_1, Φ_2 over the sets of variables V_1, V_2 , respectively, which might have a non-empty intersection. We define the sum of these formulas, $\Phi_1 + \Phi_2$, over the set of variables $V_1 \cup V_2$ by taking the union of their sets of constraint applications and merging pairs of applications that share the same constraint and the same tuple of variables, i.e., replacing the pair with a single application with a weight being the sum of the respective weights. For an integer α , the formula $\alpha \cdot \Phi$ has the same constraint applications as Φ , with weights multiplied by α .

LEMMA 4.7. *Let Γ be a non-trivial constraint language, which is neither 0-valid nor 1-valid. Then $CS(\Gamma^{T,F}, \mathbb{Z}) \leq_{ADD} CS(\Gamma, \mathbb{Z})$.*

PROOF. Given a formula $\Phi \in CS(\Gamma^{T,F}, \mathbb{Z})$, we add two auxiliary variables x_T, x_F and we translate each constraint application $\langle \hat{f}, (j_1, \dots, j_k), w \rangle$, where $\hat{f}$ is expressible by $f \in \Gamma$ with constants, into an application of f by replacing constants 0, 1 with variables x_T, x_F . Let us refer to this formula as $\Phi_1 \in CS(\Gamma, \mathbb{Z})$ and note that $|\Phi_1| = |\Phi|$ and $||\Phi_1|| = ||\Phi||$.

In the next step, we will use implementations to impose particular conditions on x_T, x_F . We refer to x_T, x_F and all the new variables introduced within the implementations as auxiliary. Let $A = O(1)$ denote their number. In the new formula, we assume that the first n variables $x_1, \dots, x_n$ are the primary variables and $x_{n+1}, \dots, x_{n+A}$ are auxiliary. For $x \in \{0, 1\}^{n+A}$ let $x|_n$ stand for the projection on the first n coordinates.

Assume first that Γ is not C-closed. Then by Lemma 2.5, point (2), we know that Γ α_1 -implements function T and α_2 -implements function F for some integers α_1, α_2 . Let $\alpha = \alpha_1 + \alpha_2$. We implement constraint applications $T(x_T)$ and $F(x_F)$, that is, we construct formulas $\Phi_T, \Phi_F \in CS(\Gamma, \mathbb{N})$ over the set of auxiliary variables, such that satisfying α constraint applications in $\Phi_T + \Phi_F$ is only possible when $x_T = 1$ and $x_F = 0$. Let $W = 2 \cdot ||\Phi|| + 1$. We define $\Phi_2 = \Phi_1 + W \cdot \Phi_T + W \cdot \Phi_F$, that is, we copy all the constraint applications from Φ_1 and add the applications from $\Phi_T + \Phi_F$ with weights multiplied by W . Recall that we have $(-||\Phi||) \leq \Phi(x) \leq ||\Phi||$ for all x . By the definition of implementation, any assignment to Φ_2 which does not satisfy $x_T = 1$ or $x_F = 0$ has value at most $(\alpha - 1) \cdot W + ||\Phi|| < \alpha \cdot W - ||\Phi||$. If the assignment x satisfies $x_T = 1, x_F = 0$, then it holds that $\Phi_2(x) = \alpha \cdot W + \Phi(x|_n) \geq \alpha \cdot W - ||\Phi||$.

Now, if Γ is C-closed, then by Lemma 2.5, point (1), Γ α -implements function XOR for some constant α . We implement XOR(x_T, x_F), that is, we construct formula $\Phi_{\text{XOR}} \in CS(\Gamma, \mathbb{N})$ over the set

of auxiliary variables, such that satisfying α constraint applications in Φ_{XOR} is only possible when $\text{XOR}(x_T, x_F) = 1$. As before, we define $\Phi_2 = \Phi_1 + W \cdot \Phi_{\text{XOR}}$, where $W = 2 \cdot \|\Phi\| + 1$. Any assignment to Φ_2 which does not satisfy $\text{XOR}(x_T, x_F)$ has value at most $(\alpha - 1) \cdot W + \|\Phi\| < \alpha \cdot W - \|\Phi\|$. For an assignment x satisfying $x_T = 1, x_F = 0$, it holds that $\Phi_2(x) = \alpha \cdot W + \Phi(x|_n) \geq \alpha \cdot W - \|\Phi\|$. It might also be the case that $x_T = 0, x_F = 1$. Then, by C-closedness we have $\Phi_2(x) = \Phi_2(\widehat{x}) = \alpha \cdot W + \Phi(\widehat{x}|_n) \geq \alpha \cdot W - \|\Phi\|$, where $\widehat{x}$ is the bit-wise complement of x .

We summarize the transformation properties for both considered cases: the new number of variables is $n + O(1)$, $|\Phi_2| = |\Phi| + O(1)$, and $\|\Phi_2\| = \|\Phi\| \cdot O(1)$. If $t < -\|\Phi\|$, then we know that all assignments x satisfy $\Phi(x) > t$ and in such case we could return an empty formula over a singleton variable set (so the only possible value is 0) and set threshold $t' = -1$: this is an equivalent instance. To see properties (4, 5) observe that, assuming $t \geq -\|\Phi\|$, $\Phi(x) = t$ (resp. $\Phi(x) \geq t$) holds for some assignment x iff. $\Phi_2(y) = t'$ (resp. $\Phi_2(y) \geq t'$) holds for some assignment y , where $t' = \alpha \cdot W + t$. $\square$

Combining the two lemmas above implies that $\text{CS}(\Gamma_1, \mathbb{Z}) \leq_{\text{ADD}} \text{CS}(\Gamma_2, \mathbb{Z})$ whenever $\deg(\Gamma_1) \leq \deg(\Gamma_2)$ and Γ_2 is neither 0-valid nor 1-valid. However, when working over the weight range $\mathbb{Z}$, we can get rid of the last assumption, which will come in useful later for proving Theorem 5.4.

LEMMA 4.8. *Let Γ_1, Γ_2 be non-trivial constraint languages such that $\deg(\Gamma_1) \leq \deg(\Gamma_2)$. Then $\text{CS}(\Gamma_1, \mathbb{Z}) \leq_{\text{ADD}} \text{CS}(\Gamma_2, \mathbb{Z})$.*

PROOF. The language Γ_2 is non-trivial so there exists some non-trivial constraint $f \in \Gamma_2$. Since f and $\neg f$ cannot be 0-valid (or 1-valid) at the same time, the language Γ_2^{NEG} is neither 0-valid nor 1-valid. We also have $\deg(\Gamma_2) = \deg(\Gamma_2^{\text{NEG}})$. First, we apply Lemma 4.6 to Γ_1 and Γ_2^{NEG} and then Lemma 4.7 to $\Gamma = \Gamma_2^{\text{NEG}}$. By transitivity we get $\text{CS}(\Gamma_1, \mathbb{Z}) \leq_{\text{ADD}} \text{CS}(\Gamma_2^{\text{NEG}}, \mathbb{Z})$. By Lemma 4.5, point (1), we obtain that $\text{CS}(\Gamma_2^{\text{NEG}}, \mathbb{Z}) \leq_{\text{ADD}} \text{CS}(\Gamma_2, \mathbb{Z})$, which proves the claim. $\square$

So far we have established a relation between Γ_1 and Γ_2 , which allows us to transform one constraint system to another by adding only a constant number of new variables. However, it works only when we allow negative weights. The next two lemmas explain how to get rid of them, so that the hardness results can be applied to the natural setting with only non-negative weights.

LEMMA 4.9. *If Γ is a non-trivial constraint language, then $\text{CS}(\Gamma, \mathbb{Z}) \leq_{\text{ADD}} \text{CS}(\Gamma^{\text{LIT}}, \mathbb{N})$.*

PROOF. For a k -ary constraint $f \in \Gamma$ and $S \subseteq [k]$, let f^S be the function expressible by f with literals given by negating variables with indices in S . For example $\text{OR}_3^{(2)}(x_1, x_2, x_3) = x_1 \vee \neg x_2 \vee x_3$.

Suppose we are given $\Phi \in \text{CS}(\Gamma, \mathbb{Z})$ on n variables. Let $\mathcal{J}_f$ denote the family of all tuples $(j_1, \dots, j_k) \in [n]^k$ that appear in some constraint application of k -ary constraint f in Φ . For each $f \in \Gamma$, we construct a formula $\Phi_f \in \text{CS}(\Gamma^{\text{LIT}}, \mathbb{N})$ over the same set of variables as Φ . The formula Φ_f consists of all constraint applications of the form $\langle f^S, (j_1, \dots, j_k), 1 \rangle$ for all $S \subseteq [k]$ and $(j_1, \dots, j_k) \in \mathcal{J}_f$.

We claim that Φ_f is a constant formula, i.e., $\Phi_f(x)$ does not depend on x . For any assignment x and k -tuple $(j_1, \dots, j_k)$, the number of sets S for which $f^S(x_{j_1}, \dots, x_{j_k}) = 1$ equals exactly $|f| = |\{s \in \{0, 1\}^k : f(s) = 1\}|$. Therefore, $\Phi_f(x) = |\mathcal{J}_f| \cdot |f|$ for any assignment x .

Let W be the absolute value of the minimum negative weight in Φ (if Φ has no negative weights, we set $W = 0$). We define $\Phi_1 = \Phi + W \cdot \sum_{f \in \Gamma} \Phi_f$. Every constraint application from Φ appears in some Φ_f , so after summing the weights we end up with only non-negative numbers. Hence $\Phi_1 \in \text{CS}(\Gamma^{\text{LIT}}, \mathbb{N})$ and $\Phi_1(x) = \Phi(x) + W \cdot \sum_{f \in \Gamma} |\mathcal{J}_f| \cdot |f|$. The second summand does not depend on x and we can set $t' = t + W \cdot \sum_{f \in \Gamma} |\mathcal{J}_f| \cdot |f|$. Since $\sum_{f \in \Gamma} |\mathcal{J}_f| \cdot |f| = O(\|\Phi\|)$, we have obtained an additive transformation with $|\Phi_1| = O(\|\Phi\|)$ and $\|\Phi_1\| = O(\|\Phi\| \cdot |\Phi|) = \|\Phi\| \cdot n^{O(1)}$. The new formula has all weights non-negative and the value of an assignment is changed by a constant. $\square$

LEMMA 4.10. *Suppose that a non-trivial Γ constraint language is neither 0-valid, 1-valid, nor 2-monotone. Then $CS(\Gamma^{LIT}, \mathbb{N}) \leq_{LIN} CS(\Gamma, \mathbb{N})$.*

PROOF. We proceed similarly to Lemma 4.7. Given a formula $\Phi \in CS(\Gamma^{LIT}, \mathbb{N})$ on n variables, for each variable x_i we introduce its negation-copy, that is, a new variable $\hat{x}_i$, and translate each constraint application $\langle \hat{f}, (j_1, \dots, j_k), w \rangle$, where $\hat{f}$ is expressible by $f \in \Gamma$ with literals, into an application of f by replacing literals $x_i, \neg x_i$ with variables $x_i, \hat{x}_i$. Let us refer to this formula on $2n$ variables as $\Phi_1 \in CS(\Gamma, \mathbb{N})$ and note that $|\Phi_1| = |\Phi|$ and $||\Phi_1|| = ||\Phi||$.

By Lemma 2.5, point (3), we know that Γ α -implements XOR for some constant α . We implement $XOR(x_i, \hat{x}_i)$, that is, for each $i \in [n]$ we construct a formula $\Phi_i \in CS(\Gamma, \mathbb{N})$ over $x_i, \hat{x}_i$ and $O(1)$ auxiliary variables (the sets of auxiliary variables are disjoint for different i), so that Φ_i can have value α only when $XOR(x_i, \hat{x}_i) = 1$. We define $\Phi_2 = \Phi_1 + W \cdot \sum_{i=1}^n \Phi_i$, where $W = ||\Phi|| + 1$. The total number of variables is $O(n)$ and we have added $O(n)$ new constraint applications. As before, let $x|_n$ denote the projection of the enlarged vector of variables to the original one.

Since the weights are non-negative, we have $0 \leq \Phi_1(x) = \Phi(x|_n) \leq ||\Phi||$ for all x . By the definition of implementation, any assignment to Φ_2 which does not satisfy $XOR(x_i, \hat{x}_i)$ for some $i \in [n]$ has value at most $(n\alpha - 1) \cdot W + ||\Phi|| < n\alpha W$. Otherwise, we have $\Phi_2(x) = n\alpha \cdot W + \Phi(x|_n) \geq n\alpha \cdot W$.

To summarize, the new number of variables is $O(n)$, $|\Phi_2| = |\Phi| + O(n)$, and $||\Phi_2|| = ||\Phi|| \cdot O(n)$. To see properties (4, 5) observe that we can assume $t \geq 0$ (by the same argument as in Lemma 4.7) and then $\Phi(x) = t$ (resp. $\Phi(x) \geq t$) holds for some assignment x iff. $\Phi_2(y) = t'$ (resp. $\Phi_2(y) \geq t'$) holds for some assignment y , where $t' = n\alpha \cdot W + t$. $\square$

In particular, all the transformation above are linear, therefore by transitivity, we can summarize them as the following corollary.

COROLLARY 4.11. *Let Γ_1, Γ_2 be non-trivial constraint languages such that $\deg(\Gamma_1) \leq \deg(\Gamma_2)$ and Γ_2 is neither 0-valid, 1-valid, nor 2-monotone. Then $CS(\Gamma_1, \mathbb{Z}) \leq_{LIN} CS(\Gamma_2, \mathbb{N})$.*

5 CONSEQUENCES FOR COMPRESSION

Having an upper bound on $\deg(\Gamma)$ already provides a compression for $\text{MAX CSP}(\Gamma, \mathbb{Z}, c)$, since we can represent all constraint applications as polynomials, sum the coefficients at all $O(n^{\deg(\Gamma)})$ monomials, and store the weights in $O(\log n)$ bits. Corollary 4.11 transforms the monomial-representation back into a small equivalent instance of $\text{MAX CSP}(\Gamma)$.

THEOREM 5.1 (FORMALIZATION OF THEOREM 1.1). *MAX CSP($\Gamma, \mathbb{N}, c$) parameterized by the number of variables n admits a compression of size $O(n^d \log n)$ for all c , where $d = \deg(\Gamma)$. Furthermore, there is a polynomial-time algorithm that reduces any instance of MAX CSP($\Gamma, \mathbb{N}, c$) to an equivalent instance of MAX CSP($\Gamma, \mathbb{N}, c + O(1)$) of size $O(n^d \log n)$. The analogous statements for MAX CSP($\Gamma, \mathbb{Z}, c$) also hold.*

PROOF. If Γ is either trivial, 0-valid, 1-valid, or 2-monotone, then $\text{MAX CSP}(\Gamma, \mathbb{N}, c)$ can be solved in polynomial time, so the compression is trivial. Suppose that it does not have any of these properties. We will prove both claims by compressing a formula $\Phi_1 \in CS(\Gamma, \mathbb{Z})$ on n variables into a formula $\Phi_2 \in CS(\Gamma, \mathbb{N})$ satisfying $|\Phi_2| = O(n^d)$.

First, we interpret each constraint application in Φ_1 as a polynomial of degree at most d . After summing these terms, we obtain a polynomial P with $O(n^d)$ monomials, each associated with an integer weight of absolute value bounded by $||\Phi_1||$. A monomial $\prod_{i=1}^k x_i$ is the characteristic polynomial for the constraint $\text{AND}_k(x_1, \dots, x_k)$, therefore P can be treated as a formula of $CS(\Gamma_{d\text{-AND}}, \mathbb{Z})$ for $\Gamma_{d\text{-AND}}$ being the language consisting of functions AND_k for all $k \leq d$. Since $\deg(\Gamma_{d\text{-AND}}) = d$,

we can apply Corollary 4.11 to obtain an equivalent formula $\Phi_2 \in \text{CS}(\Gamma, \mathbb{N})$ on $O(n)$ variables, such that $|\Phi_2| = |\Phi_1| \cdot O(1) + O(n) = O(n^d)$ and $||\Phi_2|| = ||\Phi_1|| \cdot n^{O(1)}$, so each weight can be stored in $O(\log n)$ bits. $\square$

The self-reduction in Theorem 5.1 is almost, but not quite, a kernelization: the formal decision problem that we reduce to is not the same as the original one, due to the increase of the value c . Note that by the problem definition any instance of $\text{MAX CSP}(\Gamma, \mathbb{N}, c)$ consists of at most n^c constraint applications; hence the crux of the theorem is to reduce the exponent at n .

Our lower bounds are based on reducing $\text{MAX } d\text{-CNF-SAT}$ to $\text{MAX CSP}(\Gamma, \mathbb{N})$ for $\deg(\Gamma) = d$. For $d \geq 3$ it is known that even the non-maximization variant $d\text{-CNF-SAT}$ does not admit a compression of size $O(n^{d-\varepsilon})$ [14]. However, the 2-CNF-SAT problem is solvable in polynomial time and only its maximization version becomes NP-hard. We first note that MAX 2-CNF-SAT also cannot have any non-trivial compression.

LEMMA 5.2. *$\text{MAX CSP}(\Gamma_{2\text{-SAT}}, \mathbb{N}, 3)$ does not admit a compression of size $O(n^{2-\varepsilon})$ for any $\varepsilon > 0$, unless $\text{NP} \subseteq \text{co-NP/poly}$.*

PROOF. We reduce from the VERTEX COVER problem parameterized by the number of vertices n . Here, we are given an undirected graph $G = (V, E)$ with integer k , and we are asked whether one can choose k vertices so that each edge is incident to at least one of them. VERTEX COVER is known not to admit an $O(n^{2-\varepsilon})$ -size compression, unless $\text{NP} \subseteq \text{co-NP/poly}$ [14].

Given graph G , we construct a 2-SAT formula as follows. We create a variable x_v for each vertex $v \in V$ and add a constraint application $\text{OR}_2(\neg x_v, \neg x_v)$ with a unitary weight. We also create a constraint application $\text{OR}_2(x_u, x_v)$ with weight $2n$ for each edge $uv \in E$, so the total weight of the instance is $O(n^3)$. Now, if $S \subseteq V$ is a vertex cover in G , then the assignment setting $x_v = 1$ for $v \in S$ has value $2n \cdot |E| + n - |S|$. On the other hand, if some assignment has value at least $2n \cdot |E|$, then it must satisfy all the edge-constraints and $\{v \in V : x_v = 1\}$ forms a vertex cover. Therefore there is an assignment of value at least $2n \cdot |E| + k$ iff. G has a vertex cover of size at most $n - k$.

As the parameter is preserved by the reduction, a compression of size $O(n^{2-\varepsilon})$ for $\text{MAX CSP}(\Gamma_{2\text{-SAT}}, \mathbb{N}, 3)$ implies the same for VERTEX COVER , which proves the lemma. $\square$

THEOREM 5.3 (FORMALIZATION OF THEOREM 1.2). *Let Γ be a non-trivial constraint language that is neither 0-valid, 1-valid, nor 2-monotone, and let $d = \deg(\Gamma)$. Then there is a constant c such that $\text{MAX CSP}(\Gamma, \mathbb{N}, c)$ does not admit a compression of size $O(n^{d-\varepsilon})$ for any $\varepsilon > 0$, unless $\text{NP} \subseteq \text{co-NP/poly}$.*

PROOF. First observe that a characteristic polynomial of a constraint with $d = 1$ is linear. Since this polynomial is 0/1-valued, it can depend only on one variable, which means that the constraint is 2-monotone. Hence, we can assume that $d \geq 2$.

We know that there is a constant c_d so that $\text{MAX CSP}(\Gamma_{d\text{-SAT}}, \mathbb{N}, c_d)$ does not admit a compression of size $O(n^{d-\varepsilon})$ unless $\text{NP} \subseteq \text{co-NP/poly}$: for $d = 2$ it is due to Lemma 5.2 and for $d \geq 3$ it follows from the lower bound for the non-maximization variant of $d\text{-CNF-SAT}$ [14]. As noted in Proposition 4.2, if we had such a compression for $\text{MAX CSP}(\Gamma, \mathbb{N}, c)$ with sufficiently large c , then by Corollary 4.11 it would transfer to $\text{MAX CSP}(\Gamma_{d\text{-SAT}}, \mathbb{N}, c_d)$. $\square$

Applications to specific CSPs. Having established both the lower and the upper bound, we can refer to $\deg(\Gamma)$ as the *optimal compression exponent* for $\text{MAX CSP}(\Gamma, \mathbb{N})$. We are now equipped with a handy but powerful tool for determining the optimal compressibility of $\text{MAX CSP}(\Gamma, \mathbb{N})$ as this task reduces to computing the degrees of characteristic polynomials in Γ .

As an example, we apply this technique to compute the optimal compression exponent to the following three problems, which are all NP-hard for $k \geq 2$:

- MAX k -NAE-SAT = MAX CSP($\{\text{NAE}_k\}^{LIT}, \mathbb{N}$),
- MAX Ek -LIN = MAX CSP($\{\text{XOR}\}^{NEG}, \mathbb{N}$),
- MAX k -EXACT-SAT = MAX CSP($\{\text{EX}_k\}^{LIT}, \mathbb{N}$), where $\text{EX}_k(x_1, \dots, x_k) = 1$ if and only if there is exactly one 1 in $(x_1, \dots, x_k)$.

Since $\deg(\Gamma^{LIT}) = \deg(\Gamma^{NEG}) = \deg(\Gamma)$ it suffices to analyze the characteristic polynomials for functions NAE_k , XOR_k , and EX_k . Let $e_i(x_1, \dots, x_k)$ denote i th elementary symmetric polynomial, i.e., the sum of all degree- i monomials on k variables.

$$\begin{aligned} \text{NAE}_k(x_1, \dots, x_k) &= \sum_{i=1}^{k-1} (-1)^{i-1} e_i(x_1, \dots, x_k) && \text{for odd } k, \\ \text{NAE}_k(x_1, \dots, x_k) &= \sum_{i=1}^k (-1)^{i-1} e_i(x_1, \dots, x_k) && \text{for even } k, \\ \text{XOR}_k(x_1, \dots, x_k) &= \sum_{i=1}^k (-2)^{i-1} e_i(x_1, \dots, x_k) && \text{for all } k, \\ \text{EX}_k(x_1, \dots, x_k) &= \sum_{i=1}^k i \cdot (-1)^{i-1} e_i(x_1, \dots, x_k) && \text{for all } k. \end{aligned}$$

It is easy to check these formulas using the binomial theorem. We present the argument for XOR_k as an example. Suppose, the number of 1s in the vector $(x_1, \dots, x_k)$ is ℓ . Then $e_i(x_1, \dots, x_k)$ equals $\binom{\ell}{i}$ for $i \leq \ell$ and 0 for $i > \ell$. The formula for $\text{XOR}_k(x_1, \dots, x_k)$ becomes $\sum_{i=1}^{\ell} (-2)^{i-1} \binom{\ell}{i} = (-\frac{1}{2}) \cdot (\sum_{i=0}^{\ell} (-2)^i \binom{\ell}{i} - 1) = (-\frac{1}{2}) \cdot ((-1)^{\ell} - 1)$, which is 1 for odd ℓ and 0 for even ℓ , as expected. By these identities, we deduce that the optimal compression exponent for MAX k -NAE-SAT is k in the even case, $k - 1$ in the odd case, and the optimal compression exponent for both MAX Ek -LIN and MAX k -EXACT-SAT is k .

An example of a non-symmetric constraint with a non-trivial upper bound on its degree is f_k , with $\text{AR}(f_k) = 3^k$, defined recursively: $f_0(x) = x$, and

$$f_k(x_1, \dots, x_{3^k}) = \text{NAE}_3(f_{k-1}(x_1, \dots, x_{3^{k-1}}), f_{k-1}(x_{3^{k-1}+1}, \dots, x_{2 \cdot 3^{k-1}}), f_{k-1}(x_{2 \cdot 3^{k-1}+1}, \dots, x_{3^k})).$$

Because $\deg(\text{NAE}_3) = 2$, we have $\deg(f_k) = 2^k = \text{AR}(f_k)^{\log_3(2)}$ [29].

It is tempting to seek a concise characterization of those Γ for which one can obtain a non-trivial bound on $\deg(\Gamma)$ and therefore a non-trivial compression for MAX CSP($\Gamma, \mathbb{N}$), where by non-trivial we mean a bound of the form $\deg(f) \leq \text{AR}(f) - 1$ for functions depending on all the coordinates. By generalizing the argument for NAE_k , it can be shown that $\deg(f) \leq \text{AR}(f) - 1$ when the number of vectors that satisfy f and have an even number of 1s, is equal to the number of satisfying vectors with an odd number of 1s. Unfortunately, as far as we are aware, no characterization is known describing all Γ with a non-trivial bound on $\deg(\Gamma)$, not even for symmetric polynomials induced by symmetric constraints. There exist some interesting partial results though, e.g., that if the number of variables k is a prime minus one then the degree is always k , and in general $\deg(f) \geq k - O(k^{0.548})$ [32]. On the other hand, there are infinitely many symmetric functions for which $\deg(f) = \text{AR}(f) - 3$ [32]. When it comes to non-symmetric functions, there exist infinitely many examples with $\deg(f) \leq \log(\text{AR}(f))$ [30], which is asymptotically the lowest upper bound possible [29].

Compression lower bound for negative weights. For the sake of completeness, we show that MAX CSP($\Gamma, \mathbb{Z}$) admits an analogous classification as MAX CSP($\Gamma, \mathbb{N}$), that is, whenever MAX CSP($\Gamma, \mathbb{Z}$) is NP-hard, then the upper bound from Theorem 5.1 is essentially tight. The dichotomy theorem for $\mathbb{W} = \mathbb{Z}$ can be stated in a simpler manner, as the problem becomes NP-hard whenever $\deg(\Gamma) \geq 2$ [20] and the case $\deg(\Gamma) = 1$ reduces to linear function maximization. This dichotomy also follows from the reduction below.

THEOREM 5.4. *Let non-trivial Γ be such that $d = \deg(\Gamma) \geq 2$. Then there is a constant c such that MAX CSP($\Gamma, \mathbb{Z}, c$) does not admit a compression of size $O(n^{d-\varepsilon})$ for any $\varepsilon > 0$, unless $\text{NP} \subseteq \text{co-NP/poly}$.*

PROOF. From Lemma 4.8, we know that $\text{CS}(\Gamma_{d\text{-SAT}}, \mathbb{Z}) \leq_{\text{ADD}} \text{CS}(\Gamma, \mathbb{Z})$. Therefore, an $O(n^{d-\varepsilon})$ -size compression for $\text{MAX CSP}(\Gamma, \mathbb{Z}, c)$ with sufficiently large c entails the same for $\text{MAX CSP}(\Gamma_{d\text{-SAT}}, \mathbb{Z}, c - O(1))$, which implies $\text{NP} \subseteq \text{co-NP/poly}$ (Lemma 5.2 for $d = 2$ and [14] for $d \geq 3$). $\square$

6 CONSEQUENCES FOR EXPONENTIAL-TIME ALGORITHMS

As mentioned before, our framework of reductions can be used to transfer moderately-exponential running time bounds between MAX CSP problems as well. Namely, if $\text{CS}(\Gamma_1, \mathbb{W}_1) \leq_{\text{ADD}} \text{CS}(\Gamma_2, \mathbb{W}_2)$, then an algorithm for $\text{MAX CSP}(\Gamma_2, \mathbb{W}_2, c)$ with running time $T(n)$ entails an algorithm for $\text{MAX CSP}(\Gamma_1, \mathbb{W}_1, c - O(1))$ with running time $T(n + O(1)) + n^{O(1)}$. All the constructed transformations, except for $\text{CS}(\Gamma^{\text{LIT}}, \mathbb{N}) \leq_{\text{LIN}} \text{CS}(\Gamma, \mathbb{N})$ (Lemma 4.10), are additive and in particular they work as long as negative weights are allowed. Alternatively, we can take advantage of other properties of particular constraint languages to remove the negative weights.

LEMMA 6.1. *Let $d = \deg(\Gamma) \geq 2$. Then $\text{CS}(\Gamma_{d\text{-SAT}}, \mathbb{N}) \leq_{\text{ADD}} \text{CS}(\Gamma, \mathbb{Z}) \leq_{\text{ADD}} \text{CS}(\Gamma_{d\text{-SAT}}, \mathbb{N})$, that is, these constraint systems are equivalent with respect to the relation $\leq_{\text{ADD}}$.*

PROOF. We take advantage of the fact that $\Gamma_{d\text{-SAT}}$ can express negated literals, i.e., $(\Gamma_{d\text{-SAT}})^{\text{LIT}} = \Gamma_{d\text{-SAT}}$. We have the following cycle of reductions.

$$\begin{array}{lll} \text{CS}(\Gamma_{d\text{-SAT}}, \mathbb{Z}) & \leq_{\text{ADD}} & \text{CS}(\Gamma_{d\text{-SAT}}, \mathbb{N}) \quad \text{Lemma 4.9 for } (\Gamma_{d\text{-SAT}})^{\text{LIT}} = \Gamma_{d\text{-SAT}} \\ & \leq_{\text{ADD}} & \text{CS}(\Gamma, \mathbb{Z}) \quad \text{Lemma 4.8} \\ & \leq_{\text{ADD}} & \text{CS}(\Gamma_{d\text{-SAT}}, \mathbb{N}) \quad \text{Lemma 4.8} \quad \square \end{array}$$

THEOREM 6.2. *For each $d \geq 2$ and any constant $\alpha > 1$ either all the following problems admit an $\alpha^n n^{O(1)}$ algorithm for all c , or none of them do:*

- (1) $\text{MAX CSP}(\Gamma_{d\text{-SAT}}, \mathbb{N}, c)$,
- (2) $\text{MAX CSP}(\Gamma, \mathbb{Z}, c)$ for any Γ with $\deg(\Gamma) = d$,
- (3) $\text{MAX CSP}(\Gamma, \mathbb{N}, c)$ for any Γ with $\deg(\Gamma) = d$ such that $\Gamma^{\text{NEG}} = \Gamma$ or $\Gamma^{\text{LIT}} = \Gamma$.

PROOF. As noted in Proposition 4.3, if $\text{CS}(\Gamma_1, \mathbb{W}_1) \leq_{\text{ADD}} \text{CS}(\Gamma_2, \mathbb{W}_2)$ and $\text{MAX CSP}(\Gamma_2, \mathbb{W}_2, c)$ admits an algorithm with running time $\alpha^n n^{O(1)}$, then the same holds for $\text{MAX CSP}(\Gamma_1, \mathbb{W}_1, c - O(1))$. The equivalence between (1) and (2) has been proven in Lemma 6.1. Since (2) is more general than (3), it suffices to reduce (2) into (3). Lemma 4.5, point (2), provides the reduction $\text{CS}(\Gamma, \mathbb{Z}) \leq_{\text{ADD}} \text{CS}(\Gamma^{\text{NEG}}, \mathbb{N})$ for the case $\Gamma^{\text{NEG}} = \Gamma$ and Lemma 4.9 provides the reduction $\text{CS}(\Gamma, \mathbb{Z}) \leq_{\text{ADD}} \text{CS}(\Gamma^{\text{LIT}}, \mathbb{N})$ for the case $\Gamma^{\text{LIT}} = \Gamma$. $\square$

A first corollary of this theorem is that problems of the form $\text{MAX CSP}(\Gamma, \mathbb{Z})$ are divided into equivalence classes with respect to the optimal running time. In particular, solving any $\text{MAX CSP}(\Gamma, \mathbb{Z})$ with $\deg(\Gamma) \geq 3$ in time $O(2^{(1-\varepsilon)n})$ for any $\varepsilon > 0$ contradicts the $\text{MAX 3-SAT HYPOTHESIS}$. Also, the hypothesis remains equivalent if we replace MAX 3-CNF-SAT with MAX 3-LIN SAT or MAX 3-EXACT SAT with only positive weights, because their constraint languages satisfy $\Gamma^{\text{NEG}} = \Gamma$ and $\Gamma^{\text{LIT}} = \Gamma$, respectively. Another corollary is that improving the running time $O(2^{\frac{\omega n}{3}})$ for MAX CUT or MAX DICUT with integer weights or MAX 3-NAE-SAT with positive weights would imply an analogous breakthrough for MAX 2-CNF-SAT .

Alman and Williams [1] have noted that it is not known how to improve the running time for MAX 3-CNF-SAT , even for instances with a linear number of clauses. They have, therefore, formulated a stronger hypothesis. The $\text{SPARSE MAX 3-SAT HYPOTHESIS}$ states that for any $\varepsilon > 0$ there exists $c > 0$ such that MAX 3-CNF-SAT with cn clauses does not admit an $O(2^{(1-\varepsilon)n})$ -time algorithm. Similarly, their $\text{SPARSE MAX 2-SAT HYPOTHESIS}$ states that one cannot beat running time

$O\left(2^{\frac{\omega n}{3}}\right)$ for MAX 2-CNF-SAT with cn clauses. Observe that our reductions preserve the property of having $O(n)$ different constraint applications (Condition (2) in Definition 4.1). Therefore, as long as we allow negative weights at constraint applications, we can replace MAX 2-CNF-SAT (resp. MAX 3-CNF-SAT) in this hypothesis with MAX CSP($\Gamma, \mathbb{Z}$) for any constraint language Γ of degree 2 (resp. 3) to obtain an equivalent statement.

7 CONCLUSIONS AND OPEN PROBLEMS

We have provided a complete characterization of the optimal compression for MAX CSP(Γ) in the case of a Boolean domain. A natural question arises about larger domains. Our approach does not transfer even to the case with a domain of size 3, since there is no unique way to represent functions $\{0, 1, 2\}^k \rightarrow \{0, 1\}$ as polynomials. One may consider, e.g., embedding to a Boolean domain or using non-multilinear polynomials, but it is not clear, which approach leads to the optimal degree and how to find accompanying lower bounds.

On the exponential-time front, we have shown that MAX d -CNF-SAT is as hard as any MAX CSP of degree d as long as negative weights are allowed. Although we were able to get rid of the latter assumption in several cases, there is still a gap in this classification: does improving the running time for any degree- d MAX CSP($\Gamma, \mathbb{N}$) imply an improvement for MAX d -CNF-SAT?

REFERENCES

- [1] Josh Alman and Virginia Vassilevska Williams. 2020. OV graphs are (probably) hard instances. In *Proceedings of the 11th Innovations in Theoretical Computer Science Conference, ITCS 2020 (Leibniz International Proceedings in Informatics (LIPIcs '20), Vol. 151)*, Thomas Vidick (Ed.). Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany, 83:1–83:18. DOI: <https://doi.org/10.4230/LIPIcs.ITCS.2020.83>
- [2] Noga Alon, Gregory Gutin, Eun Jung Kim, Stefan Szeider, and Anders Yeo. 2011. Solving MAX-r-SAT above a tight lower bound. *Algorithmica* 61, 3 (2011), 638–655. DOI: <https://doi.org/10.1007/s00453-010-9428-7>
- [3] David A. Mix Barrington, Richard Beigel, and Steven Rudich. 1994. Representing Boolean functions as polynomials modulo composite numbers. *Computational Complexity* 4, 4 (1994), 367–382. DOI: <https://doi.org/10.1007/BF01263424>
- [4] Richard Beigel. 1993. The polynomial method in circuit complexity. In *Proceedings of the Eighth Annual Structure in Complexity Theory Conference*. IEEE Computer Society, 82–95. DOI: <https://doi.org/10.1109/SCT.1993.336538>
- [5] Karl Bringmann, Nick Fischer, and Marvin Künnemann. 2019. A fine-grained analogue of Schaefer’s Theorem in P: Dichotomy of Exists^k-forall-quantified first-order graph properties. In *Proceedings of the 34th Computational Complexity Conference, CCC 2019 (Leibniz International Proceedings in Informatics (LIPIcs '19), Vol. 137)*, Amir Shpilka (Ed.). Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany, 31:1–31:27. DOI: <https://doi.org/10.4230/LIPIcs.CCC.2019.31>
- [6] Andrei A. Bulatov. 2017. A dichotomy theorem for nonuniform CSPs. In *Proceedings of the 58th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2017*, Chris Umans (Ed.). IEEE Computer Society, 319–330. DOI: <https://doi.org/10.1109/FOCS.2017.37>
- [7] Andrei A. Bulatov, Martin Grohe, Phokion G. Kolaitis, and Andrei A. Krokhin (Eds.). 2009. *The Constraint Satisfaction Problem: Complexity and Approximability*, 25.10. - 30.10.2009. Dagstuhl Seminar Proceedings, Vol. 09441. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Germany. Retrieved from <http://drops.dagstuhl.de/portals/09441/>
- [8] Jin-Yi Cai and Xi Chen. 2017. Complexity of counting CSP with complex weights. *J. ACM* 64, 3 (2017), 19:1–19:39. DOI: <https://doi.org/10.1145/2822891>
- [9] Hubie Chen, Bart M. P. Jansen, and Astrid Pieterse. 2020. Best-case and worst-case sparsifiability of Boolean CSPs. *Algorithmica* 82, 8 (2020), 2200–2242. DOI: <https://doi.org/10.1007/s00453-019-00660-y>
- [10] Nadia Creignou. 1995. A dichotomy theorem for maximum generalized satisfiability problems. *J. Comput. Syst. Sci.* 51, 3 (1995), 511–522. DOI: <https://doi.org/10.1006/jcss.1995.1087>
- [11] Nadia Creignou, Sanjeev Khanna, and Madhu Sudan. 2001. *Complexity Classifications of Boolean Constraint Satisfaction Problems*. SIAM monographs on discrete mathematics and applications, Vol. 7. SIAM. DOI: <https://doi.org/10.1137/1.9780898718546>
- [12] Víctor Dalmau, Andrei A. Krokhin, and Rajsekar Manokaran. 2018. Towards a characterization of constant-factor approximable finite-valued CSPs. *J. Comput. Syst. Sci.* 97, 1 (2018), 14–27. DOI: <https://doi.org/10.1016/j.jcss.2018.03.003>
- [13] Vladimir Deineko, Peter Jonsson, Mikael Klasson, and Andrei Krokhin. 2008. The approximability of MAX CSP with fixed-value constraints. *J. ACM* 55, 4, Article 16 (Sept. 2008), 37 pages. DOI: <https://doi.org/10.1145/1391289.1391290>

- [14] Holger Dell and Dieter van Melkebeek. 2014. Satisfiability allows no nontrivial sparsification unless the polynomial-time hierarchy collapses. *J. ACM* 61, 4 (2014), 23:1–23:27. DOI : <https://doi.org/10.1145/2629620>
- [15] Fedor V. Fomin, Daniel Lokshtanov, Saket Saurabh, and Meirav Zehavi. 2019. *Kernelization: Theory of Parameterized Preprocessing*. Cambridge University Press. DOI : <https://doi.org/10.1017/9781107415157>
- [16] Gregory Z. Gutin and Anders Yeo. 2017. Parameterized constraint satisfaction problems: A survey. In *The Constraint Satisfaction Problem: Complexity and Approximability*. Andrei A. Krokhin and Stanislav Zivny (Eds.). Dagstuhl Follow-Ups, Vol. 7. Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik, 179–203. DOI : <https://doi.org/10.4230/DFU.Vol7.15301.7>
- [17] Bart M. P. Jansen and Astrid Pieterse. 2017. Sparsification upper and lower bounds for graph problems and not-all-equal SAT. *Algorithmica* 79, 1 (2017), 3–28. DOI : <https://doi.org/10.1007/s00453-016-0189-9>
- [18] Bart M. P. Jansen and Astrid Pieterse. 2019. Optimal sparsification for some binary CSPs using low-degree polynomials. *TOCT* 11, 4 (2019), 28:1–28:26. DOI : <https://doi.org/10.1145/3349618>
- [19] Bart M. P. Jansen, Shivesh K. Roy, and Michal Włodarczyk. 2021. On the hardness of compressing weights. In *Proceedings of the 46th International Symposium on Mathematical Foundations of Computer Science, MFCS 2021 (LIPIcs, Vol. 202)*. Filippo Bonchi and Simon J. Puglisi (Eds.). Schloss Dagstuhl—Leibniz-Zentrum für Informatik, 64:1–64:21. DOI : <https://doi.org/10.4230/LIPIcs.MFCS.2021.64>
- [20] Peter Jonsson and Andrei A. Krokhin. 2007. Maximum H -colourable subdigraphs and constraint optimization with arbitrary weights. *J. Comput. Syst. Sci.* 73, 5 (2007), 691–702. DOI : <https://doi.org/10.1016/j.jcss.2007.02.001>
- [21] Sanjeev Khanna, Madhu Sudan, Luca Trevisan, and David P. Williamson. 2000. The approximability of constraint satisfaction problems. *SIAM J. Comput.* 30, 6 (2000), 1863–1920. DOI : <https://doi.org/10.1137/S0097539799349948>
- [22] Stefan Kratsch, Daniel Marx, and Magnus Wahlström. 2016. Parameterized complexity and kernelizability of max ones and exact ones problems. *TOCT* 8, 1 (2016), 1:1–1:28. DOI : <https://doi.org/10.1145/2858787>
- [23] Stefan Kratsch and Magnus Wahlström. 2010. Preprocessing of min ones problems: A dichotomy. In *Proceedings of the 37th International Colloquium on Automata, Languages and Programming, ICALP 2010 (Lecture Notes in Computer Science, Vol. 6198)*. Samson Abramsky, Cyril Gavoille, Claude Kirchner, Friedhelm Meyer auf der Heide, and Paul G. Spirakis (Eds.). Springer, 653–665. DOI : https://doi.org/10.1007/978-3-642-14165-2_55
- [24] Andrei A. Krokhin and Stanislav Zivny (Eds.). 2017. *The Constraint Satisfaction Problem: Complexity and Approximability*. Dagstuhl Follow-Ups, Vol. 7. Schloss Dagstuhl—Leibniz-Zentrum fuer Informatik. Retrieved from <http://www.dagstuhl.de/dagpub/978-3-95977-003-3>
- [25] Victor Lagerkvist and Magnus Wahlström. 2020. Sparsification of SAT and CSP problems via tractable extensions. *ACM Trans. Comput. Theory* 12, 2 (2020), 13:1–13:29. DOI : <https://doi.org/10.1145/3389411>
- [26] Andrea Lincoln, Virginia Vassilevska Williams, and Ryan Williams. 2018. Tight hardness for shortest cycles and paths in sparse graphs. In *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2018 (New Orleans, Louisiana) (SODA'18)*. Society for Industrial and Applied Mathematics, 1236–1252. DOI : <https://doi.org/10.1137/1.9781611975031.80>
- [27] Konstantin Makarychev and Yuri Makarychev. 2017. Approximation algorithms for CSPs. In *The Constraint Satisfaction Problem: Complexity and Approximability*. Andrei Krokhin and Stanislav Zivny (Eds.). Dagstuhl Follow-Ups, Vol. 7. Schloss Dagstuhl—Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany, 287–325. DOI : <https://doi.org/10.4230/DFU.Vol7.15301.287>
- [28] M. Minsky and S. Papert. 1969. *Perceptrons*. MIT Press, Cambridge, MA.
- [29] Noam Nisan and Mario Szegedy. 1994. On the degree of Boolean functions as real polynomials. *Computational Complexity* 4, 4 (1994), 301–313. DOI : <https://doi.org/10.1007/BF01263419>
- [30] Hans Ulrich Simon. 1982. A tight $\Omega(\log \log n)$ -bound on the time for parallel RAM's to compute nondegenerated Boolean functions. *Information and Control* 55, 1-3 (1982), 102–106. DOI : [https://doi.org/10.1016/S0019-9958\(82\)90477-6](https://doi.org/10.1016/S0019-9958(82)90477-6)
- [31] Gábor Tardos and David A. Mix Barrington. 1998. A lower bound on the mod 6 degree of the OR function. *Computational Complexity* 7, 2 (1998), 99–108. DOI : <https://doi.org/10.1007/PL00001597>
- [32] Joachim von Zur Gathen and James R. Roche. 1997. Polynomials with two values. *Combinatorica* 17, 3 (01 Sep 1997), 345–362. DOI : <https://doi.org/10.1007/BF01215917>
- [33] R. Ryan Williams. 2007. *Algorithms and Resource Requirements for Fundamental Problems*. Ph. D. Dissertation. Carnegie Mellon University, USA. Advisor(s) Blum, Manuel. AAI3274191.
- [34] Chee-Keng Yap. 1983. Some consequences of non-uniform conditions on uniform classes. *Theor. Comput. Sci.* 26, 3 (1983), 287–300. DOI : [https://doi.org/10.1016/0304-3975\(83\)90020-8](https://doi.org/10.1016/0304-3975(83)90020-8)
- [35] Dmitriy Zhuk. 2020. A proof of the CSP dichotomy conjecture. *J. ACM* 67, 5 (2020), 30:1–30:78. DOI : <https://doi.org/10.1145/3402029>

Received 6 October 2021; accepted 14 September 2023